

Data Structures – Unit 13

Bucharest University of Economic Studies

B-Trees



B-Tree - Definition

A B-Tree is a tree in which a total order relation is implemented, each node containing a set of values with the following restriction:

$$\text{MINIMUM} \leq n \leq \text{MAXIMUM}$$

where:

$$\text{MAXIMUM} = 2 * \text{MINIMUM}$$



B-Trees - Characteristics

Search trees based on key order:

- Binary search trees: one search key for each node;
- Multiple search paths trees: multiple search keys for each node according to the order of the tree;



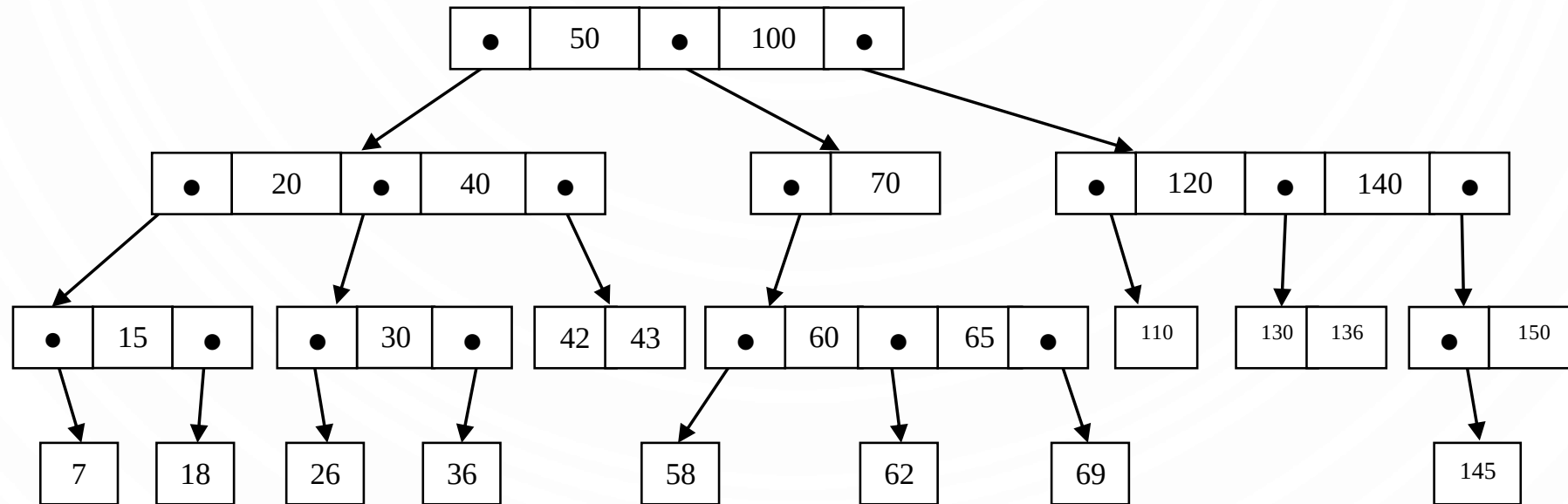
B-Trees - Characteristics

- a more general approach on binary search trees;
- each node has: a number of maximum m keys strictly ordered, with $m+1$ subtrees;
- the number of keys differs for each node;
- each node is called a page;
- B-Trees have a high **branching factor** which means that the trees are *fat* with a relatively small height;



B-Trees - Characteristics

Multiple search paths tree



B-Trees - Characteristics

Node structure:

n	P₀	K₀	P₁	K₁	...	...	P_{n-2}	K_{n-2}	P_{n-1}	K_{n-1}	P_n
----------	----------------------	----------------------	----------------------	----------------------	-----	-----	------------------------	------------------------	------------------------	------------------------	----------------------

-P₀, P₁, ..., P_{n-1} pointers to subtrees;

-K₀, K₁, ..., K_{n-2} key values;



B-Trees - Characteristics

B-Tree of m order – properties:

- For each node the possible number of direct descendants, k , is:

$$\left\lceil \frac{m}{2} \right\rceil \leq k \leq m$$

- If the degree of a BTree is m , then the maximum number of keys is $m-1$;
- Each node can have at most m descendants, if $m-1$ is the number of keys in that particular node;



B-Trees - Characteristics

The min height of a B-Tree, h , is :

$$h = \lceil \log_m(n + 1) \rceil - 1$$

where:

m – the order of the tree greater than or equal to 2;

n – the number of keys in the tree.



B-Trees - Characteristics

The max height of a B-Tree, h , is :

$$h = \left\lceil \log_{\left\lfloor \frac{m}{2} \right\rfloor} \left(\frac{n+1}{2} \right) \right\rceil$$

where:

m – the order of the tree greater than or equal to 2;

n – the number of keys in the tree.



B-Trees - Implementation

Database Management Systems:

Search index operations by using a pair of values

<key, article address> for fast access to data;

Storing records in ascending order for fast
lookups;



B-Tree Operations

Search operation:

- start from the root;
- compare the search key, x , with each key from the current node;
 - $c_i < x < c_{i+1}$ search in node P_i ;
 - $x > c_n$ search in node P_n ;
 - $x < c_0$ search in node P_0 .



B-Tree Operations

2. Insertion of a key:

- preceded by a search operation;
- if the key is found, stop insert operation;
- if the key is not found, search ended in a leaf node where the new key is inserted;



B-Tree Operations

- Insertion scenarios:

- The leaf node has at most $m-2$ keys:

- The key is inserted, and the structure remains as it is;

- The leaf node has the maximum number of keys $m-1$:

- “fission”: the leaf node is splitted in half, operation called ramification after which two new nodes are created;



B-Tree Operations

B-Tree order 5:

- Maximum number of keys: 4;
- Inserting key values: 22, 57, 41, 59.

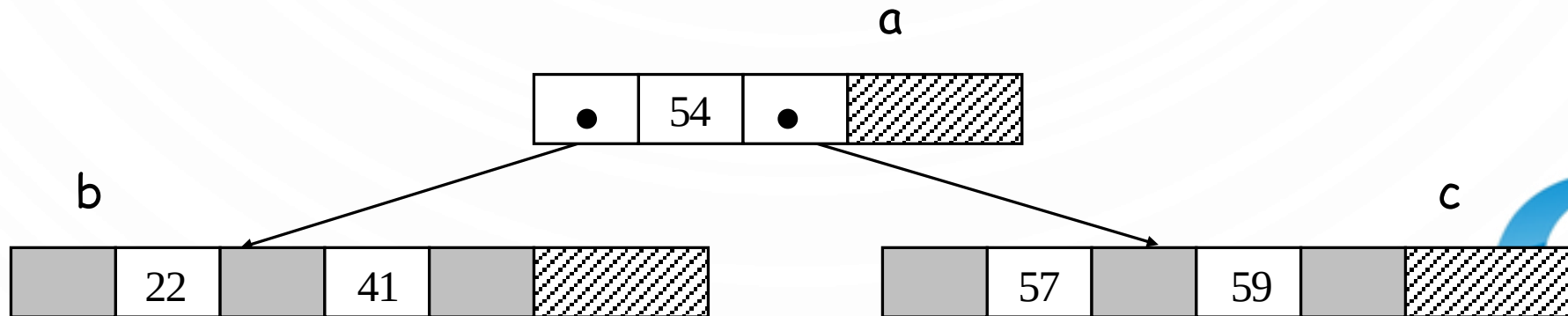
	22		41		57		59	
--	----	--	----	--	----	--	----	--



B-Tree Operations

When inserting key 54, the root node exceeds the maximum number of keys, and the node will be splitted (fission);

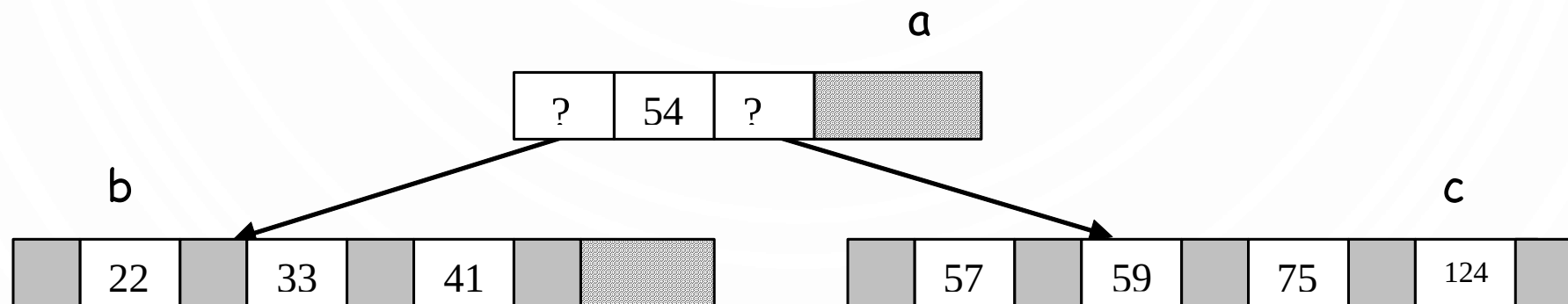
A new root is created which doesn't adhere to the restriction of a B-Tree;



B-Tree Operations

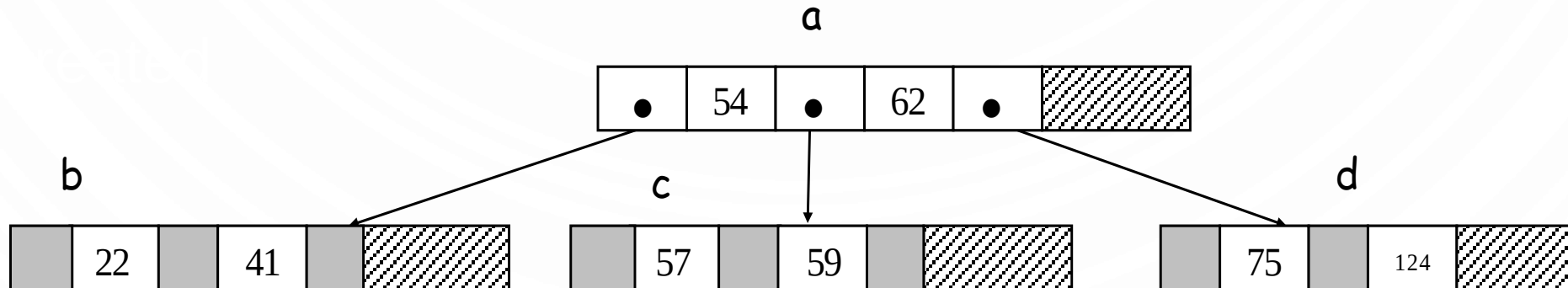
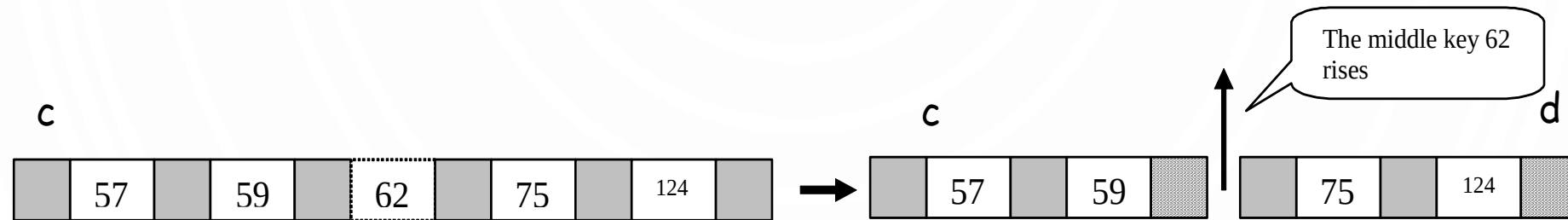
The structure is inserting the following key values:

33, 75, 124



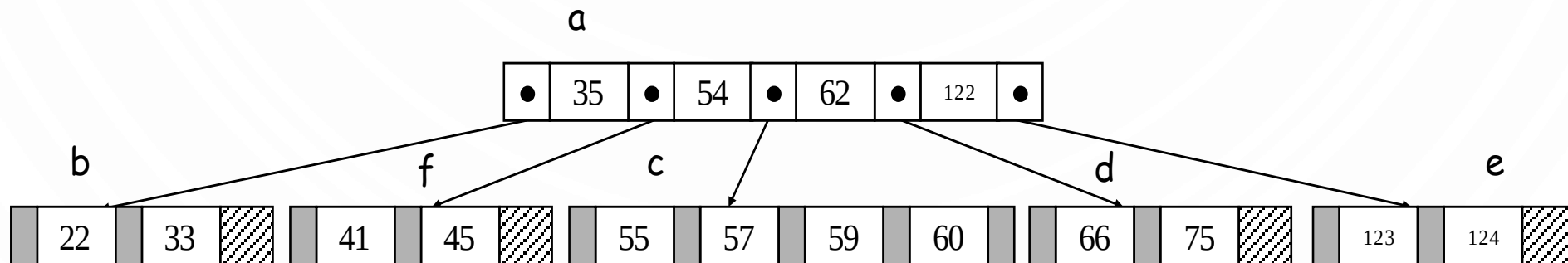
B-Tree Operations

Inserting the key 62: a new fission on the node **c**:



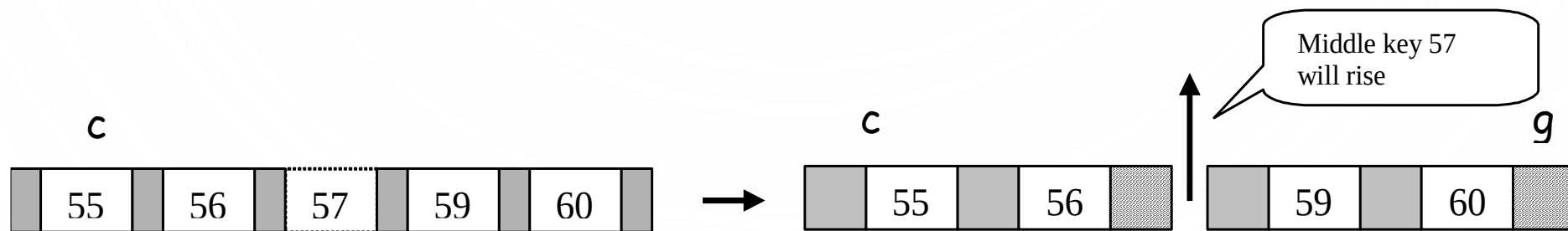
B-Tree Operations

Inserting new keys: 33, 122, 123, 55, 60, 45, 66, 35



B-Tree Operations

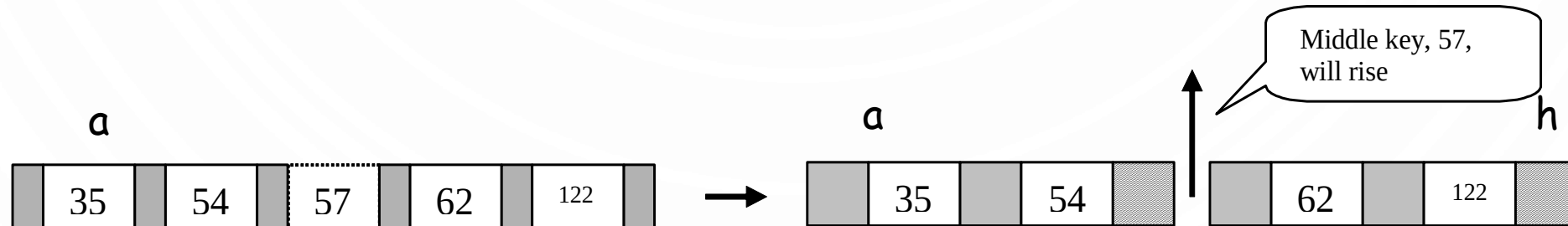
Inserting key 56: will split the *c* node, with the middle key, 57, rising to an upper level;



B-Tree Operations

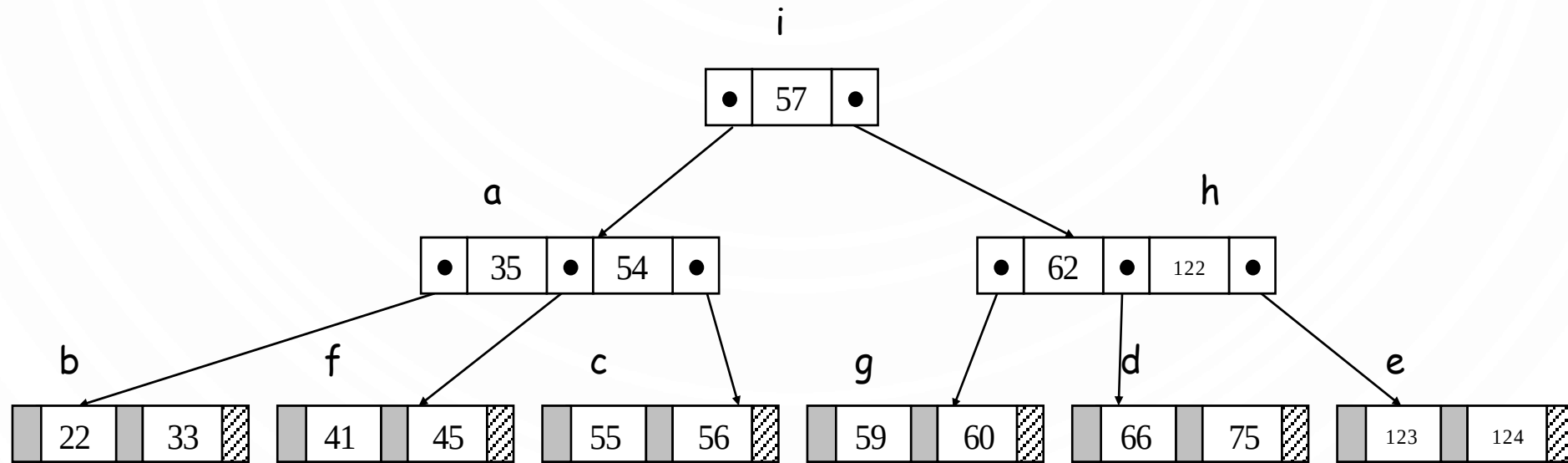
The upper node is full, so it is also split and a new root is created.

The fission algorithm is again applied for the root, node a.



B-Tree Operations

The new structure of the B-Tree



B-Tree Operations

Insertion algorithm in a B-Tree:

- Start from the root;
- Search for the key until is found (if it's not the search process stops on a leaf);
- If the leaf is overpopulated:
 - divide the leaf into two separated nodes and advance the middle key;
 - Otherwise insert the key into the leaf node;



B-Tree Operations

Worst case scenario:

The fission algorithm is applied on each level of the tree, starting from the leaf in which the key was inserted and ending on the root;



B-Tree Operations

3. Deletion of a key in a B-Tree:

- Simple if the search key is in a leaf;
- Complex if the search key is not in a leaf:
 - Logical deletion with another key closest in order with the search key which is in a leaf node;



B-Tree Operations

- Deletion scenarios:

- The node contains at least $\left\lceil \frac{m}{2} \right\rceil + 1$ keys: a simple deletion from an array of sorted values;
- The node has the minimum number of keys $\left\lceil \frac{m}{2} \right\rceil$:
 - After deletion the number of keys is insufficient for complying with the condition of a B-Tree:
 - **Sharing** between the current node and a neighbor with at least $\left\lceil \frac{m}{2} \right\rceil + 1$ keys;
 - **Fusion** between two nodes: the current node which has $\left\lceil \frac{m}{2} \right\rceil$



B-Tree Operations

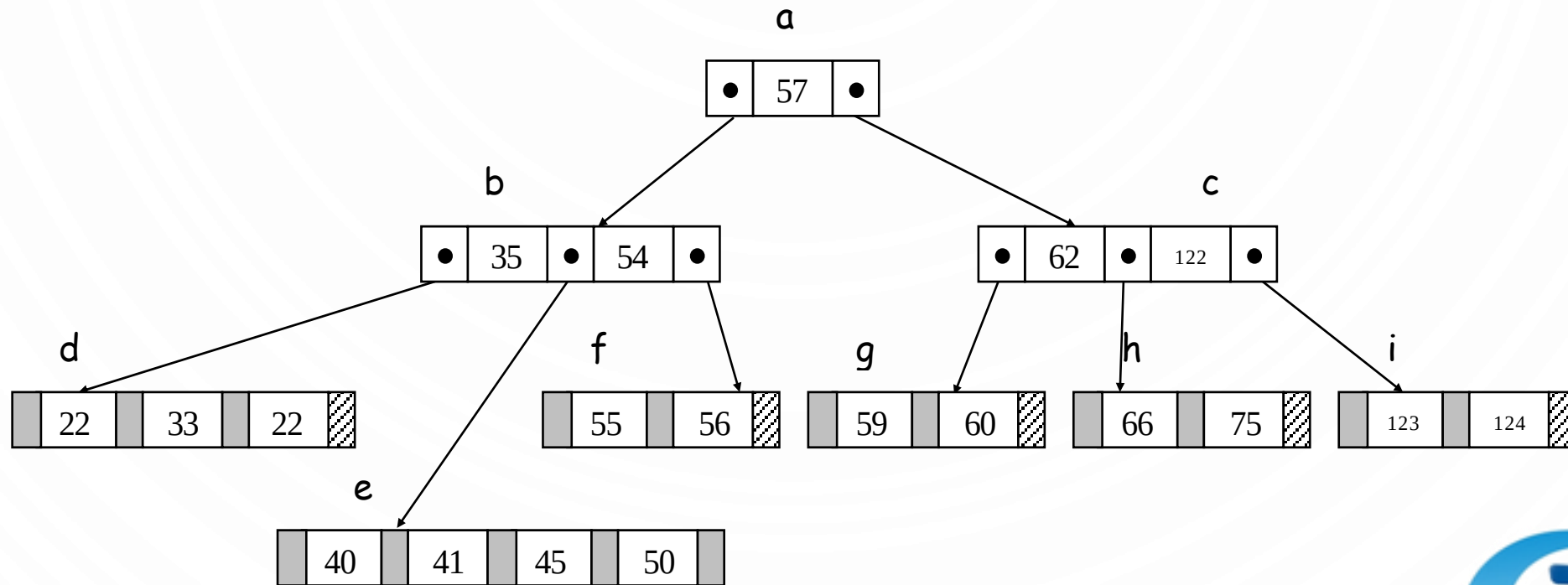
Worst case scenario:

The sharing of fusion operations reaches the top and another root is created;



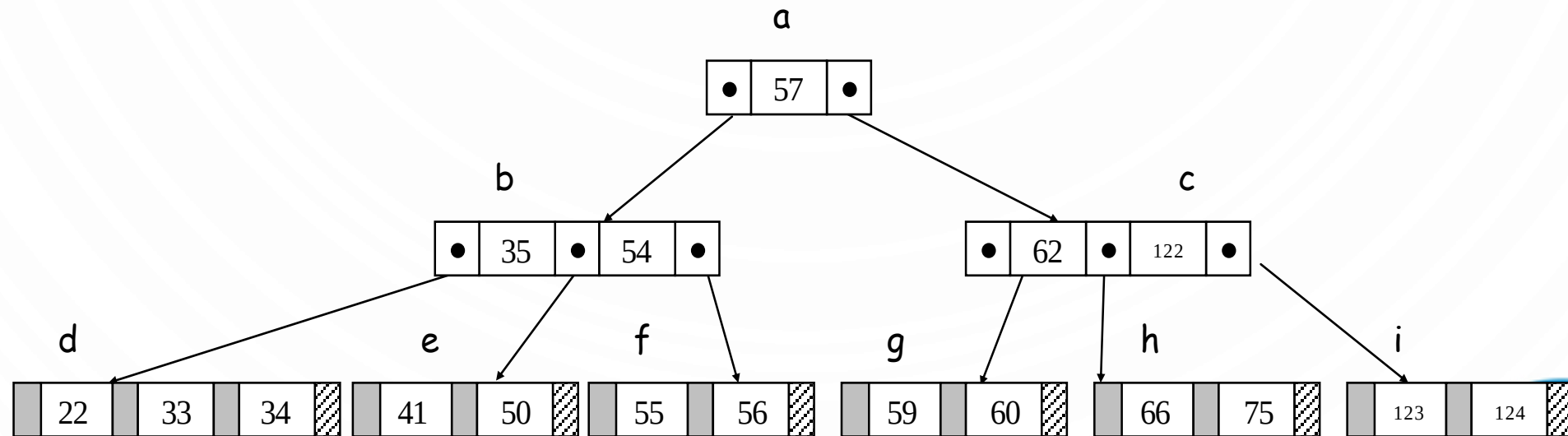
B-Tree Operations

A B-Tree of order 5:



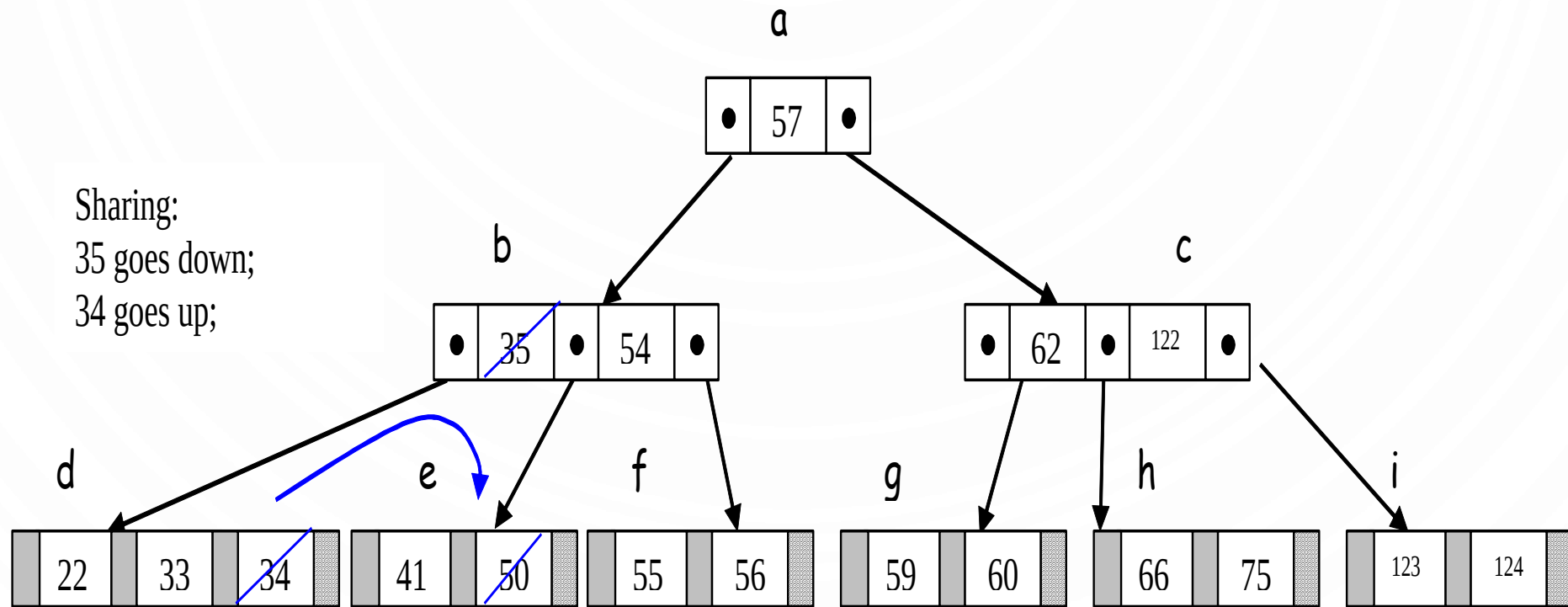
B-Tree Operations

Deleting the values 40 and 45:



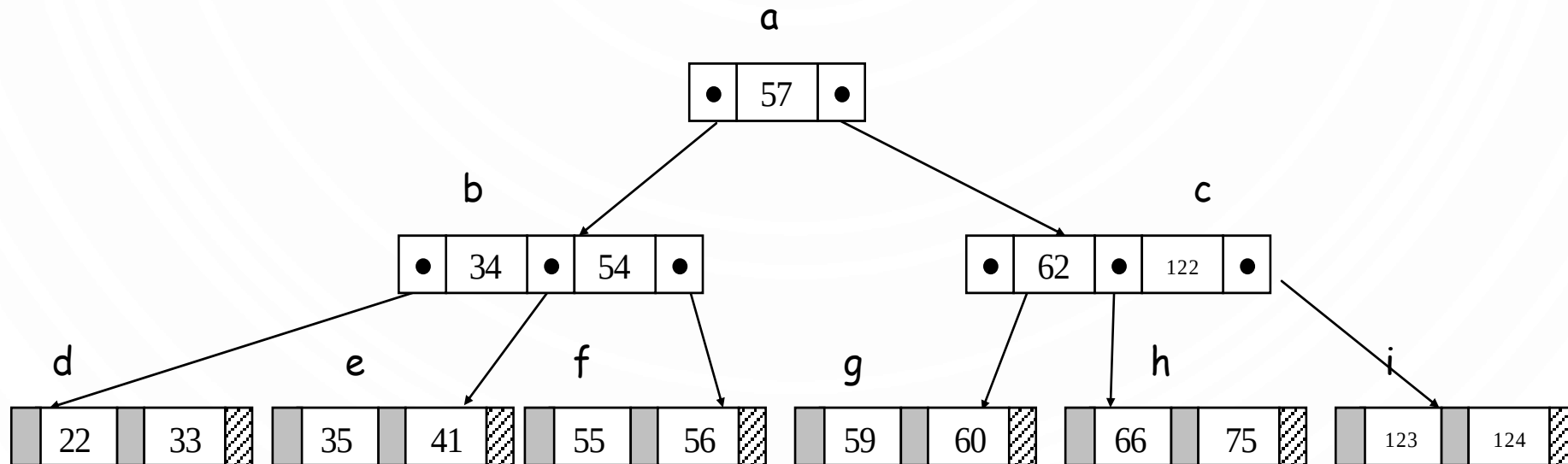
B-Tree Operations

Deleting the key value 50: sharing between d and e;



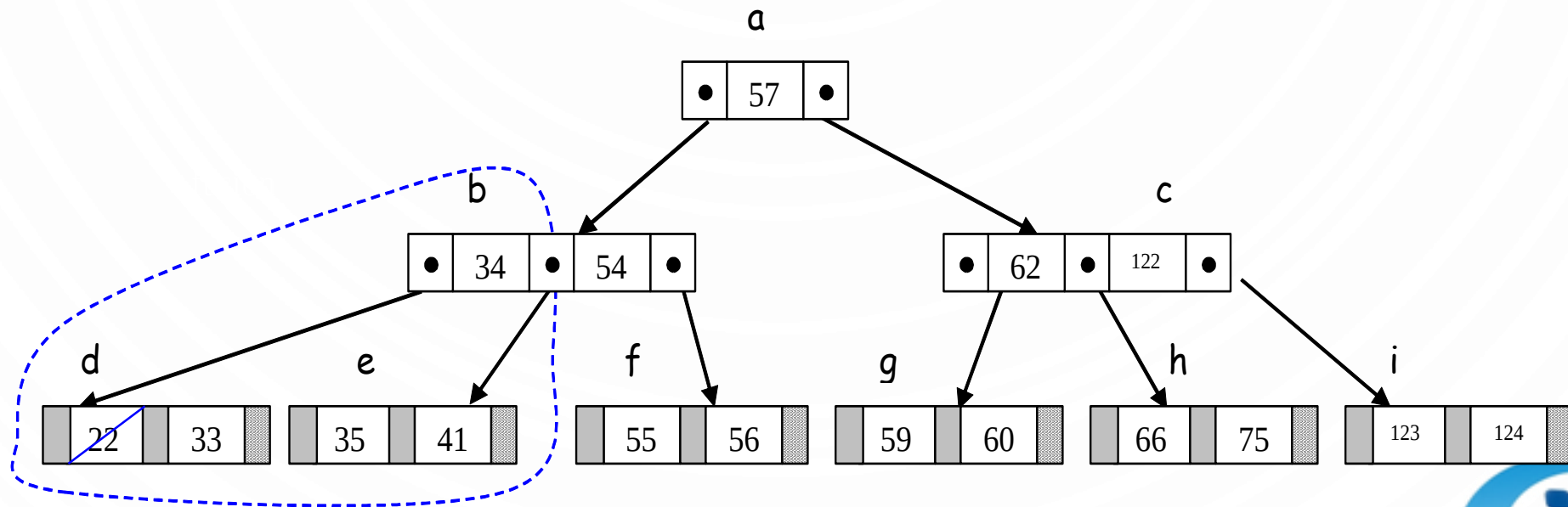
B-Tree Operations

The B-Tree structure after deletion



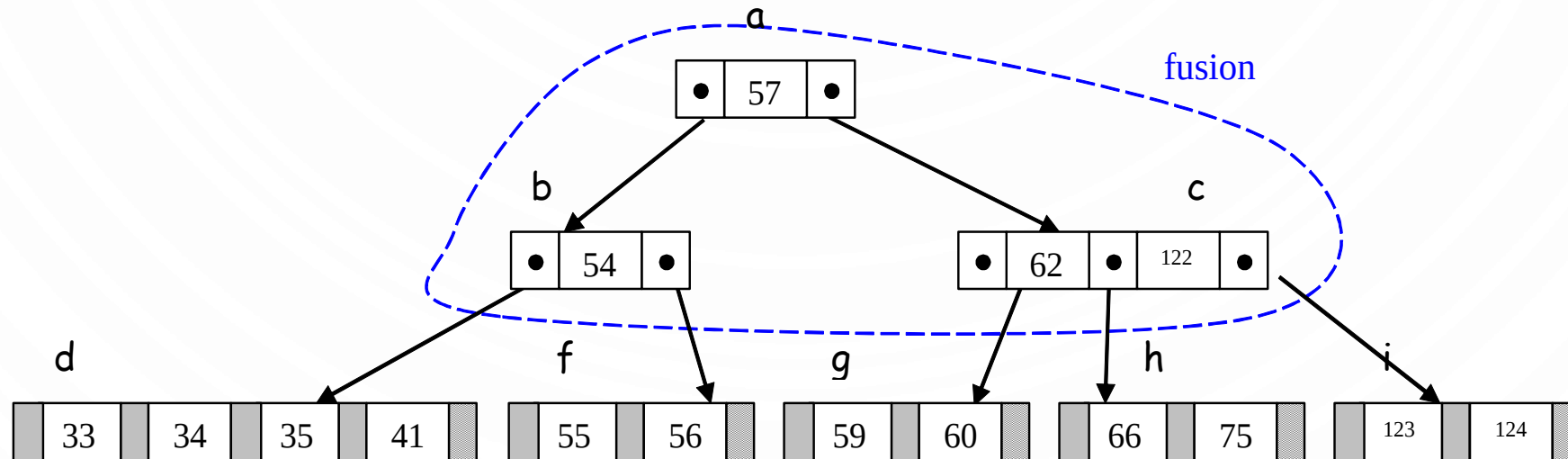
B-Tree Operations

Deleting the key 22: fusion between d and e.



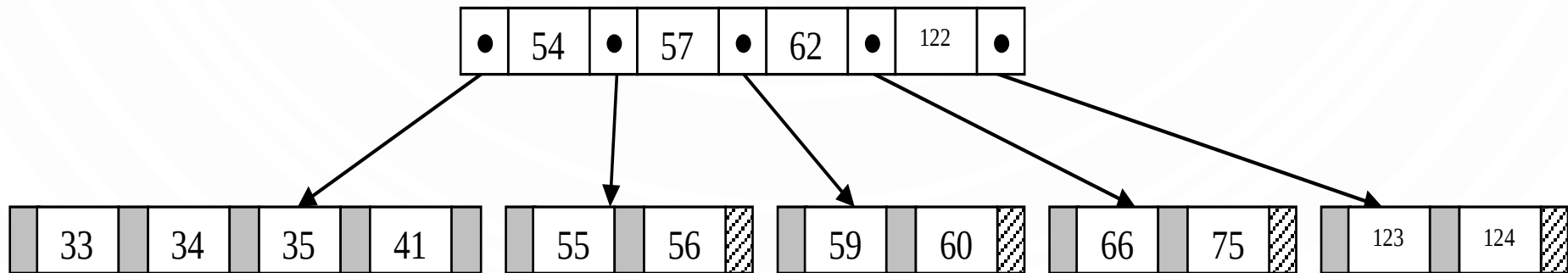
B-Tree Operations

After d and e have fused, the node b hasn't the minimum number of keys so it will fuse again with the creation of a new root;



B-Tree Operations

The final structure of the B-Tree after deleting the key 22



B-Tree Operations

The algorithm for deleting a node from a B-Tree:

- Start from the root and search for the key until is found;
- If the key is found:
 - Delete the search key;
 - Check the minimum number of elements after deletion
 - If the B-Tree condition isn't checked try to share a key or otherwise try to fuse two nodes by taking one key from the parent;
 - Do this until each page comply with the B-Tree condition;



References

- Ion IVAN, Marius POPA, Paul POCATILU, coordonatori – *Structuri de date*, Editura ASE, Bucuresti, 2008, ISBN 978-606-505-031-0
- Ion SMEUREANU – *Programarea in limbajul C/C++*, Editura CISON, 2001
- Michael MAIN – *Data Structures & Other Objects Using Java*, Editura Pearson, 2012, ISBN: 9780132911504
- <http://www.acs.ase.ro>
- <http://www.itcsolutions.eu>

